

The digital-to-analog converter and data transfer between peripherals

1 Paper's purpose

By the end of this first laboratory every student should have basic knowledge regarding the digital-to-analog converter and the timers included on the SoC (System-on-Chip).

2 Introduction

In this chapter will be presented:

- 2.1. Additional information about the working modes of the ADC2 module
- 2.2. The digital-to-analog conversion system
- 2.3. Timers
- 2.4. Data transfer between peripherals

2.1 Additional information about the working modes of the ADC2 module

The ADC2 converter is based on the running principle of a successive approximation register (SAR), each conversion being performed using a clock that is derived from the system clock. Depending on the configuration of the 7-3 bits of the ADC2CF register (AD2SC), the division ratio between the system clock and the SAR clock is set according to the equation:

$$CLK_{SAR2} = \frac{SYSCLK}{AD2SC + 1}$$

where: SYSCLK is the system clock,

AD2SC is the decimal value of the number formed by the 7-3 bits of the ADC2CF register.

However, the maximum SAR frequency may not exceed 7.5 MHz.

The converter can track the input signal continuously (as has been done so far), or only when a conversion occurs, for the rest of the time it will not be working. These modes can be configured with the AD2TM bit in AD2CN:

- **continous tracking mode:** the converter is always active and follows the input signal, and the conversion can be started as soon as the command is received (eg setting the AD2BUSY bit) and takes 8 SAR clock cycles.
- **low-power mode:** as long as the device is not converting, it operates in a low-power mode, in which it consumes very little power. This working mode is very efficient for systems where low power consumption is desired. However, upon receiving a conversion command, it takes 3 clock cycles for the device to start converting. In this case, the time required for a conversion increases from 8 clocks to 11 SAR clock cycles, as can be seen in Fig. 2.1.

corresponding latches except until a timer overflow occurs. Selecting the timer used for the conversion is configured through the bits 4 and 3 of DAC0CN register.

2.2.2 Scaling the converter output voltage

In some cases, input data in the converter must be rearranged before writing. This operation requires additional instructions, which increase the size of the executable code and reduce the speed of the DAC. In order to overcome this shortcoming, the programmer has the possibility to modify the location of the information in the two data registers of the converter using the bits 0, 1 and 2 of the DAC0CN configuration register. The values of these bits corresponding to the different data sets are shown in Fig. 2.2.2. For more information on the DAC0CN definition, refer to the microcontroller documentation (file C8051F04xRev1_4.pdf, p. 108).

Bits2-0: DAC0DF2-0: DAC0 Data Format Bits:

000: The most significant nibble of the DAC0 Data Word is in DAC0H[3:0], while the least significant byte is in DAC0L.

DAC0H								DAC0L							
				MSB											LSB

001: The most significant 5-bits of the DAC0 Data Word is in DAC0H[4:0], while the least significant 7-bits are in DAC0L[7:1].

DAC0H								DAC0L							
				MSB										LSB	

010: The most significant 6-bits of the DAC0 Data Word is in DAC0H[5:0], while the least significant 6-bits are in DAC0L[7:2].

DAC0H								DAC0L							
				MSB										LSB	

011: The most significant 7-bits of the DAC0 Data Word is in DAC0H[6:0], while the least significant 5-bits are in DAC0L[7:3].

DAC0H								DAC0L							
							MSB							LSB	

1xx: The most significant 8-bits of the DAC0 Data Word is in DAC0H[7:0], while the least significant 4-bits are in DAC0L[7:4].

DAC0H								DAC0L							
							MSB							LSB	

Fig. 2.2.2. Layout Modes

2.3 Timer 3

Timer 3 is a 16-bit timer/counter implemented with two special registers: TMR3L (stores the lower byte) and TMR3H (stores the upper byte).

Depending on the T3M1 and T3M0 bits from TMR3CF register, the timer can be synchronized with one of the following clock sources:

- system clock,
- system clock with frequency divided by 2 or 12,
- an external oscillator with frequency divided by 8,
- falling fronts of the T3 pin signal (capture mode only).

The timer can act as a counter by counting the falling fronts of an external signal applied to T3 pin (for this, the bit C/T3 in TMR3CN register must be set). The timer can count ascending or descending (default Timer 3 counts ascending).

Timer 3 is enabled by setting the TR3 bit in the TMR3CN register.

The CP/RL3 bit in TMR3CN register sets the timer mode:

- mode 0 – auto-reload
- mode 1 – capture mode

For more information about TMR3CN definition and TMR3CF definition, refer to the microcontroller documentation (file C8051F04xRev1_4.pdf, p. 301, p. 302).

2.3.1 Auto-Reload mode

The auto-reload mode is selected by resetting the CP/RL3 bit in TMR3CN register. In this mode, Timer 3 can be set to start counting in ascending or descending order. By default, it will count ascending. When an overflow occurs (transition 0xFFFF - 0x0000):

- TMR3L and TMR3H registers will be reloaded with the values stored in the RCAP3L and RCAP3H registers respectively. The counting does not resume from 0x0000, but from the value stored in these registers.
- TF3 bit in TMR3CN register is set. If the interruptions associated with this timer are enabled, setting this bit will cause an interruption.
- EXF3 bit in TMR3CN register changes its logic value. This bit can be considered the most significant bit of a 17-bit counter.

Timer 3 reloading (of TMR3L and TMR3H counting registers) can also be ordered by the user. To do this, set the EXEN3 bit in TMR3CN register and reset DCEN bit from TMR3CF register. With these configurations being made, each falling front of the signal applied by the user on the T3EX pin will cause a reload. Note: if the DCEN bit has a logic value of 1, Timer 3 will count ascending, or descending if the signal applied to the T3EX pin has a logic value of 1 (or if 0, the transitions of this signal will not cause reload).

For more information on the RCAP3L definition and RCAP3H definition, refer to the microcontroller documentation (file C8051F04xRev1_4.pdf, p. 303).

2.3.2 Capture mode

The capture mode can be selected by setting the CP/RL3 bit in TMR3CN register. In addition, the EXEN3 bit in the same register must be set to activate the snapshots (captures). This mode is used if the timer value should be stored at a specific moment of time specified by the user. When Timer 3 is configured in capture mode, the falling front of the signal applied to the T3EX pin determines:

- storing the values of TMR3L and TMR3H registers in RCAP3L and RCAP3H registers respectively.
- setting the EXF3 bit in TMR3CN register. If the interruptions associated with this timer are enabled, setting this bit will cause an interruption.

By default, the timer counts ascending, but the counting can be changed by setting the DCEN bit in TMR3CF register. An overflow (transition 0xFFFF - 0x0000, counting ascending, or a transition 0x0000 - 0xFFFF, counting descending) determines the setting of the TF3 bit in TMR3CN register. If the interruptions associated with this timer are enabled, setting this bit will cause an interruption.

Note: All configuration registers, including reloading and counting ones, are on page 1 of the SFR registers.

3 Application example

3.1 Requirements

Implement a system that transmits the voice signal captured by a microphone to a speaker.

3.2 Proposed solutions

3.2.1 Hardware modules description

We will briefly present the implementation of a system that uses the C8051F040 development board. Additionally, two external modules are used:

- input module
 - the role of this module is to amplify the signal from the microphone, as it has a very small amplitude (millivolt order) and must be protected against noise.
 - it contains an audio signal amplification circuit for the microphone signal and a low pass filter.
 - it connects to the microcontroller through two wires; one is the mass of the system, and the other transmits the audio input signal.
 - the input module diagram is shown in Fig. 3.2.1.a.

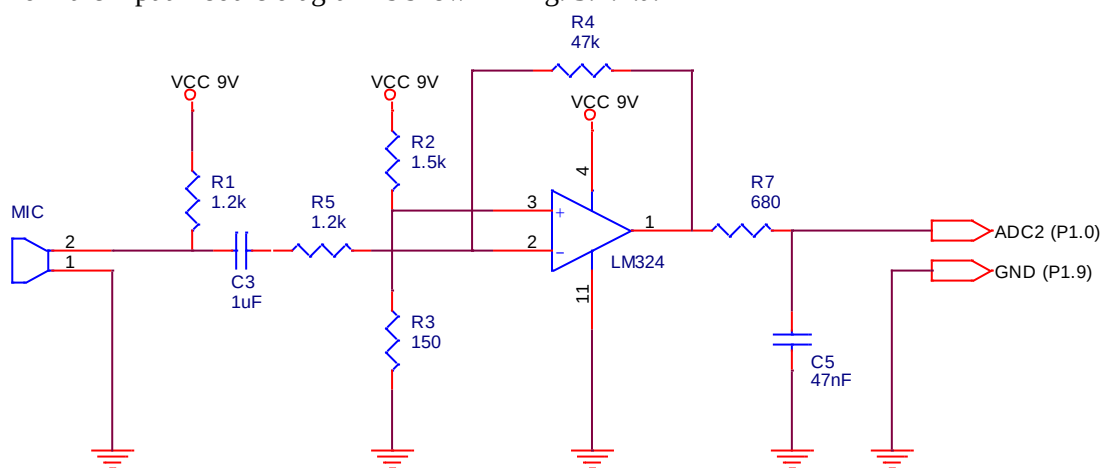


Fig. 3.2.1.a. Input Amplifier

Signals provided by a conventional power microphone can not be used directly from the converter because the voltage generated is in the millivolt range. In addition, ADC2 converts only positive voltages. For these reasons, the signal provided by the microphone is passed through an amplification stage with an operational amplifier with a negative reaction and fixed gain.

Since the amplifier is powered by a unipolar source, it will have to raise the continuous level of the amplified vocal signal. Since the range of converted values is between 0V and 2.4V, the DC current level will be set to 1.2V so that the voice signal range will fit within the limits mentioned above. At the amplifier output, an anti-aliasing low pass filter is connected (made up of resistor R7 and capacitor C5) to limit the bandwidth input to 4 kHz. A 2.4V reference voltage will be used to get a signal-to-noise ratio as large as possible.

- output module
 - the role of this module is to increase the current at the output of the DAC0 converter, so that it can command a load of ohms or tens of ohms, typical to multimedia headphones. In this case, the headphone impedance is 32 ohms.
 - the module contains a power amplifier with high current capability, two low pass filters and a resistive divider.
 - the output module diagram is shown in Fig. 3.2.1.b.

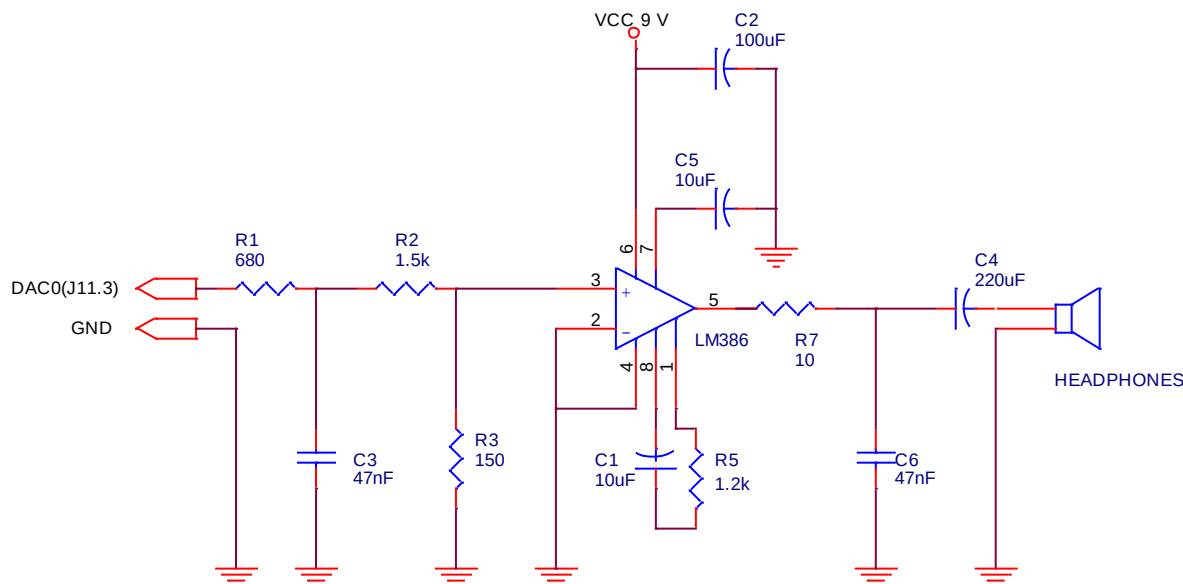


Fig 3.2.1.b. Output amplifier

One of the filters (consisting of R1 and C3) is placed immediately after the converter to eliminate the quantization noise (to eliminate high frequencies and to mitigate the sudden jumps from the signal supplied by the converter) and the second one (consisting of R7 and C6) located after the amplifier, to reduce its own noise. The resistive divider (R2 and R3) is used to reduce the input signal amplitude and eliminate the risk that the output of the amplifier will be brought to the supply voltage.

3.2.2 Algorithm description

Note that the algorithm and the source code follow the general structure presented in Paper 1, chapter 3.2.2.

The solution presented will be composed of two routines: `main` and `init`.

The algorithm consists of a loop that calls itself infinitely. This loop has two stages:

- the voltage supplied by the input module is converted to an 8-bit digital value.
- the converted result is transferred to DAC0H (the most significant 8 bits of the total of 12 bits of DAC0).

The main routine (`main`) calls the initialization subroutine (`init`) then passes into the infinite loop area labeled `start`. The area labeled `start` respects the steps of the above algorithm and then recalls itself.

The initialization subroutine:

- disables the interrupt system
 - please see Paper 1 for details.
- disables the watchdog timer mechanism
 - please see Paper 1 for details.
- enables the crossbar
 - please see Paper 1 for details.
- configures port P1
 - please see Paper 2 for details.
- configures the reference voltage
 - please see Paper 2 for details.
- configures the analog-to-digital converter
 - please see Paper 2 for details.
- configures the digital-to-analog converter
 - the converter is activated by setting the DAC0EN bit.
 - the output voltage update time of the DAC0 is configured as the time of writing in the DAC0H register.
 - the data format of DAC0 is configured as in the last variant of Fig. 2.2.2.

- for more information about the DAC0CN definition refer to the microcontroller documentation (file C8051F04xRev1_4.pdf, p. 108).

Note: This algorithm is very simple, but provides very little control over the sampling rate and the amount of information transferred. The sampling rate of the analog voice signal is determined exclusively by the clock frequency at which the successive approximation register (SAR) works, thereby this effectively determines the duration of a conversion, as well as the number of clocks of the system required for executing each instruction from the `start` loop.

The flow chart of the program is presented in Fig. 3.2.2.a.

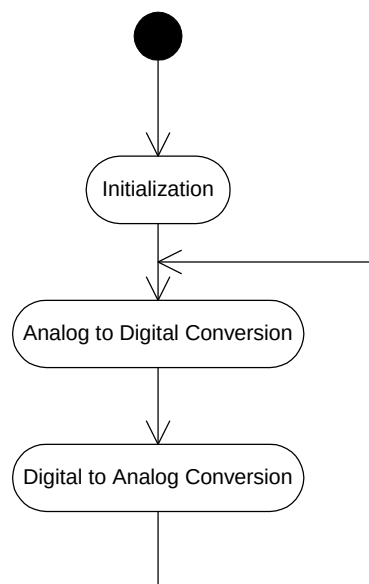


Fig. 3.2.2.a. Flow chart of AD-DAConversion_v1

For direct control over the amount of information transferred, we propose another solution where the conversion of the signal from the microphone and the transfer to the headphones is done periodically, with a period determined by a timer overflow.

The structure of the program is maintained, with the following changes:

- in the loop labeled `start` an additional stage is introduced where it is expected a Timer 3 overflow
- Timer 3 is also set in the initialization subroutine
 - the auto-reload mode is configured with increment frequency equal to clock frequency (~ 3.06MHz)
 - to obtain a sampling frequency of 16KHz (this frequency was chosen to be higher than the sampling limit frequency) the reload value 0xFF44 (value stored in RCAP3H and RCAP3L) is calculated according to the formula:

$$F_s = \frac{F_{CK}}{65536}$$

The flow chart of the program is presented in Fig. 3.2.2.b.

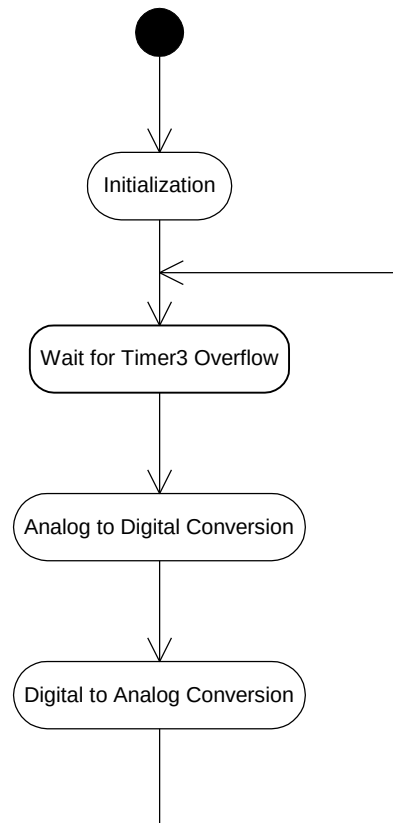


Fig. 3.2.2.b. Flow chart of AD-DAConversion_v2

4 Exercises

1. Create a new project using the Z:\Lab8051\Laborator4\AD-DAConversion_v1.asm file following the steps outlined in Paper 1, Section 2.2.
2. Complete the initialization subroutine code to disable interruptions, watchdog timer and to enable crossbar, port P1 configuration, internal reference voltage, ADC2 analogue-to-digital converter. Indication: follow the explanations presented in Paper 2, Section 2.1 and ANNEX 5.1.
3. Execute the application by pressing the Go button in the IDE button bar and observe system behavior.
4. Create a new project using the Z:\Lab8051\Laborator4\AD-DAConversion_v2.asm file following the steps outlined in Paper 1, Section 2.2.
5. Complete the initialization subroutine code to disable interruptions, watchdog timer and to enable crossbar, port P1 configuration, internal reference voltage, ADC2 analogue-to-digital converter. Indication: follow the explanations presented in Paper 2, Section 2.1 and ANNEX 5.1.
6. Execute the application by pressing the Go button in the IDE button bar and observe system behavior.
7. Modify the program AD-DAConversion_v2.asm so that the voice signal sampling is done at the limit of the Nyquist condition. Calculate the values to be entered in RCAP3, to observe the under sampling effects.
8. Edit a program that generates a rectangular signal.
9. Edit a program that generates a saw tooth signal.
10. Edit a program that generates a triangular signal.
11. Edit a program that modifies the turning on/ turning off frequency of the LED according to the amplitude of the voice signal.
12. Edit a program that modulates the amplitude of the LED brightness according to the amplitude of the voice signal.
13. Edit a program which, using the keypad, generates the corresponding DO gamma values with the DAC0 according to the following correspondence: the a key – low DO note, the s key – RE note, the d key – MI key, the f key – FA note, the g key - SOL note, the h key – LA note, the j key - SI note and the k key – up DO note. The frequencies corresponding to each note are found in ANNEX 5.3.
14. Modify the AD-DAConversion_v1.asm program so that the transfer is on 12 bits without changing the SFR page.
15. Edit a program to transfer information from both ADCs to a single DAC.

5 Appendixes

5.1 AD-DAConversion_v1 code

```

;-----
;
;
; FILE NAME   : AD-DAConversion_v1.asm
; TARGET MCU  : C8051F040
; DESCRIPTION : Data transfer from ADC2 to DAC0.
;
; NOTES:
;-----

        $include (c8051f040.inc)                ; Include register definition file.

;-----
; RESET and INTERRUPT VECTORS
;-----

                cseg        AT 0x0000            ; Reset Vector
                ljmp        main                 ; Locate a jump to the start of code
                                                ;at the reset vector.

;-----
; MAIN PROGRAM CODE SEGMENT
;-----

mainCodeSeg    segment        CODE

                rseg        mainCodeSeg          ; Switch to this code segment.
                using      0                     ; Specify register bank for the following
                                                ; program code.

main:          acall        init                 ; Initialization of all used SFR's

start:         mov         SFRPAGE, #ADC2_PAGE   ; Starting the algorithm
                                                ; Use SFRs on the ADC2 Page
                clr         AD2INT               ; Reset AD2INT.
                setb        AD2BUSY              ; Force ADC2 to start a new conversion
                jnb         AD2INT, $             ; Wait for ADC2 to finish the conversion
                mov         A, ADC2              ; Write the converted value in the accumulator

                mov         SFRPAGE, #DAC0_PAGE   ; Use SFRs on the DAC0 Page
                mov         DAC0H, A             ; Write the value from the accumulator
                                                ; in upper byte of DAC0
                sjmp        start                 ; Start over again

;-----
; FUNCTION CODE
;-----

init:          ...                ; Disable global interrupts
                ...                ; Disable Watch Dog Timer

initIOandCross: ...                ; Use SFRs on the configuration Page
                ...                ; Enable Crossbar
                ...                ; Configure P1.0 as analog input
                ...                ; For analog data the port must be set as
                ...                ; open-drain.

initVREF:      ...                ; Use SFRs on the 0x00 Page
                ...                ; ADC2 voltage reference from internal VREF
                ...                ; Enable Bias Generator

initADC2:      ...                ; Use SFRs on the ADC2 Page
                ...                ; Set AIN1.0 as single-ended input
                ...                ; Select AIN1.0 as input channel for
                ...                ; the analog multiplexer
                ...                ; Configure a *1 Gain
                ...                ; Enable ADC2; Continuous tracking;

```

```

initDAC0:      mov     SFRPAGE, #DAC0_PAGE      ; Use SFRs on the DAC0 Page
               mov     DAC0CN, #0x84           ; Enable DAC0
               ;a conversion will start only after a write
               mov     DAC0L, #0x00           ;in DAC0H; the data format used is lxx.

               ret

;-----
; End of file.

END
    
```

5.2 AD-DAConversion_v2 code

```

;-----
;
;
; FILE NAME   : AD-DAConversion_v2.asm
; TARGET MCU  : C8051F040
; DESCRIPTION : Data transfer from ADC2 to DAC0.
;
; NOTES:
;
;-----

    $include (c8051f040.inc)                ; Include register definition file.

;-----
; RESET and INTERRUPT VECTORS
;-----

               cseg     AT 0x0000             ; Reset Vector
               ljmp     main                 ; Locate a jump to the start of code
               ;at the reset vector.

;-----
; MAIN PROGRAM CODE SEGMENT
;-----

mainCodeSeg    segment      CODE

               rseg     mainCodeSeg          ; Switch to this code segment.
               using    0                    ; Specify register bank for the following
               ; program code.

main:          acall     init                ; Initialization of all used SFR's

start:         ; Starting the algorithm
               mov     SFRPAGE, #TMR3_PAGE   ; Use SFRs on the Timer3 Page
               jnb     TF3, $                 ; Waiting for a Timer3 overflow.
               clr     TF3                    ; Reset the overflow flag.

               mov     SFRPAGE, #ADC2_PAGE    ; Use SFRs on the ADC2 Page
               clr     AD2INT                 ; Reset AD2INT.
               setb    AD2BUSY                ; Force ADC2 to start a new conversion
               jnb     AD2INT, $              ; Wait for ADC2 to finish the conversion
               mov     A, ADC2                ; Write the converted value in the accumulator

               mov     SFRPAGE, #DAC0_PAGE    ; Use SFRs on the DAC0 Page
               mov     DAC0H, A               ; Write the value from the accumulator
               ;in upper byte of DAC0
               sjmp     start                 ; Start over again

;-----
; FUNCTION CODE
;-----

init:          ...                          ; Disable global interrupts
               ...                          ; Disable Watch Dog Timer

initIOandCross: ...                        ; Use SFRs on the configuration Page

               ...                          ; Enable Crossbar
               ...                          ; Configure P1.0 as analog input
               ...                          ; For analog data the port must be set as
               ...                          ; open-drain.
    
```

```
initVREF:      ...      ; Use SFRs on the 0x00 Page
               ...      ; ADC2 voltage reference from internal VREF
               ...      ; Enable Bias Generator

initADC2:      ...      ; Use SFRs on the ADC2 Page
               ...      ; Set AIN1.0 as single-ended input
               ...      ; Select AIN1.0 as input channel for
               ...      ; the analog multiplexer
               ...      ; Configure a *1 Gain
               ...      ; Enable ADC2; Continuous tracking;

initDAC0:      mov       SFRPAGE, #DAC0_PAGE      ; Use SFRs on the DAC0 Page
               mov       DAC0CN, #0x84            ; Enable DAC0
               ...      ; a conversion will start only after a write
               mov       DAC0L, #0x00             ; in DAC0H; the data format used is 1xx.

initTimer3:    mov       SFRPAGE, #TMR3_PAGE      ; Use SFRs on the Timer3 Page
               mov       TMR3CF, #0x08            ; Counting frequency is the SYSCLK frequency
               mov       RCAP3H, #0xFF            ; Set the reload value
               mov       RCAP3L, #0x44            ; The reload value triggers a sampling
               mov       TMR3H, #0xFF            ; frequency of 16kHz. See the formula!
               mov       TMR3L, #0x44
               mov       TMR3CN, #0x04            ; Enable Timer3.

               ret

;-----
; End of file.

END
```

5.3 Corresponding frequencies of notes from Do gamma

The piano has eight notes in an octave, each note being generated by pressing one of the keys a, s, d, f, g, h, j, k. Notes have the following frequencies:

Do – 55Hz
Re – 61.735Hz
Mi – 65.406Hz
Fa – 73.416Hz
Sol – 82.407Hz
La – 87.307Hz
Si – 97.999Hz
Do – 110Hz